

Unidad 2

Historia y Componentes de los Sistemas Operativos

Objetivos de la Unidad

- 1 Definir objetivos, funciones y evolución histórica de los Sistemas Operativos
- 2 Identificar los componentes fundamentales de un SO moderno
- 3 Analizar problemas clásicos que resuelven los SO
- 4 Comparar arquitecturas y tipos de SO actuales
- 5 Contextualizar el rol del SO en sistemas contemporáneos (nube, móviles, edge)

Evolución Histórica: De los Mainframes a la Nube

Década	Tecnología	SO Representativos	Característica Clave
1950s	Mainframes	GM-NAA I/O, IBSYS	Procesamiento por lotes
1960s	Tiempo compartido	CTSS, MULTICS	Multiprogramación
1970s	Minicomputadoras	UNIX, CP/M	Portabilidad, kernel en C
1980s	Computadoras personales	MS-DOS, Mac OS	Interfaz de usuario
1990s	Cliente-servidor	Windows NT, Linux	Redes, 32/64 bits
2000s	Móviles/Web	Windows XP, Android, iOS	Conectividad, touch
2010s	Cloud/Contenedores	Windows 10, Docker, K8s	Virtualización ligera
2020s	Edge/IoT	Windows 11, ROS 2, RHEL	Distribuido, tiempo real

Referencia: Tanenbaum, A. y Bos, H. Modern Operating Systems, 5th ed., Pearson, 2023, Cap. 1

¿Qué es un Sistema Operativo?

1

Administrador de recursos

Gestiona CPU, memoria, almacenamiento y dispositivos; asigna recursos de manera eficiente y justa.

2

Máquina virtual / extendida

Abstrae la complejidad del hardware y provee interfaces uniformes a los programas.

3

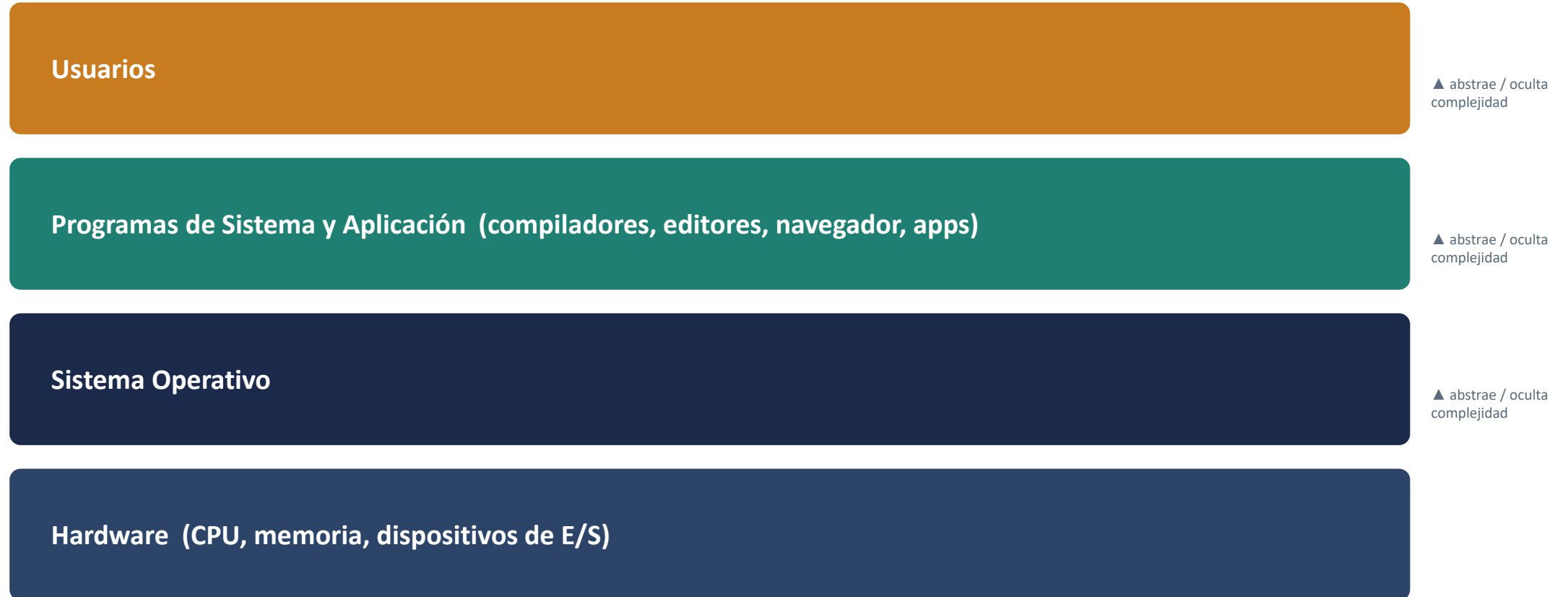
Controlador / proveedor de servicios

Ofrece servicios comunes a las aplicaciones y garantiza operación confiable y segura.

“El SO es el software que hace usable el hardware”

— Silberschatz, A., et al. Operating System Concepts, 11th ed., 2022

El SO como Intermediario: Visión en Capas

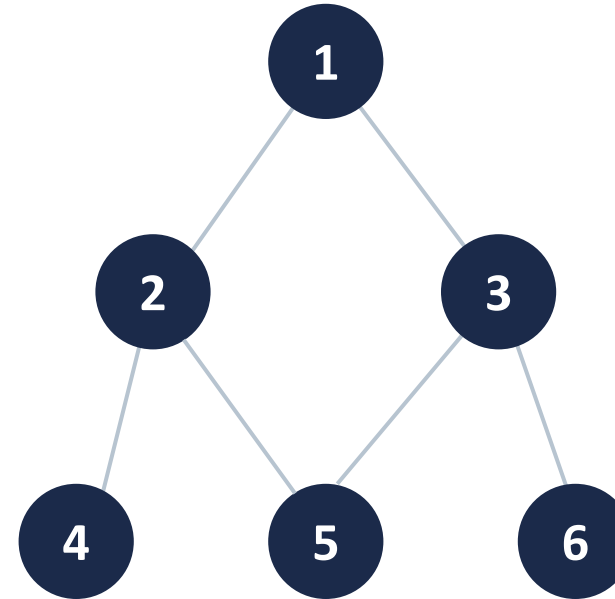


Cada capa consume únicamente los servicios de la capa inferior y ofrece servicios a la capa superior.

Adaptado de Silberschatz, Galvin & Gagne (2014) y Tanenbaum & Bos (2014)

¿Qué Problemas Resuelve un SO? — Concurrency

- Un programa en ejecución es, en realidad, un conjunto de procesos en ejecución.
- ¿Cómo sabe el proceso 2 que el proceso 1 ya terminó?
- Si 2 y 3 se pueden ejecutar en paralelo, ¿quién se ejecuta primero?
- Si 2 y 3 ya terminaron, ¿cómo se enteró el proceso 5 de este suceso?



¿Qué se deduce al analizar la ilustración?

¿Qué Problemas Resuelve un SO? — Asignación de Recursos

- Los procesos requieren recursos para ejecutarse (CPU, memoria, dispositivos).
- Si dos programas se ejecutan en paralelo, ¿en qué orden se deben asignar los recursos a sus procesos?
- Mientras un programa no termina, sus procesos asociados no pueden liberar los recursos.
- Una propiedad deseable de un SO es la multiprogramación: mantener varios procesos listos para aprovechar la CPU al máximo.

Memoria

Prog1

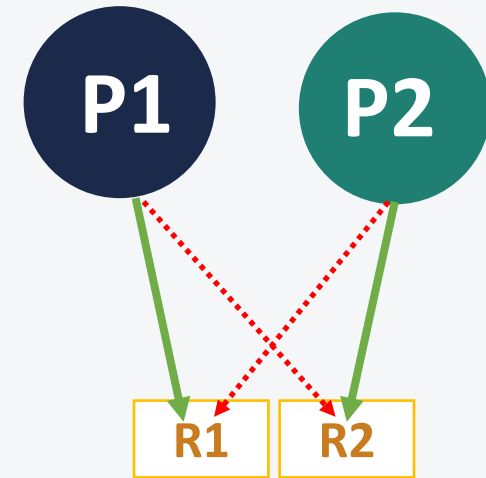
init → 2 → 1,3

Prog2

init → 4,5

¿Qué Problemas Resuelve un SO? — Bloqueo Mutuo

- También llamado interbloqueo, traba mortal, deadlock o abrazo mortal.
- Se presenta cuando el procesador no puede asignar un recurso compartido a un conjunto de procesos pendientes de terminar.
- Cada proceso (o hilo) tiene un recurso asignado, pero requiere el recurso asignado al otro proceso para continuar.



P1 tiene R1 y espera R2 · P2 tiene R2 y espera R1 → espera circular

COMPONENTES

Componentes Principales de un SO Moderno



Procesos



Memoria



E/S



Archivos



Red



Seguridad



1. Gestor de Procesos

Proceso = Programa en ejecución + estado + recursos

Responsabilidades

- Creación / destrucción (fork / exec / exit)
- Scheduling (planificación)
- Sincronización (semáforos, mutex)
- Comunicación (pipes, sockets, memoria compartida)

Ejemplo moderno: Contenedores (namespaces, cgroups)

CONEXIÓN CON UNIDAD 1

El multiproceso y el cómputo heterogéneo CPU/GPU/NPU vistos en Unidad 1 dependen de este gestor para decidir dónde y cuándo ejecutar cada tarea.

2. Gestor de Memoria



Jerarquía gestionada: Registros → Caché L1/L2/L3 → RAM → SSD/NVMe → Cloud Storage

Responsabilidades

- Asignación / liberación
- Paginación / segmentación
- Memoria virtual (swap)
- Caché y buffers

Ejemplo moderno: Memoria no volátil (Optane), DAX

CONEXIÓN CON UNIDAD 1

Es la misma jerarquía de memoria estudiada en la Unidad 1 — aquí se detalla cómo el SO la administra.

3. Gestor de E/S



Arquitectura en capas: Aplicación → API de E/S → Drivers genéricos → Drivers específicos → Hardware

Responsabilidades

- Buffering y caching
- Scheduling de discos
- Gestión de interrupciones
- Soporte plug-and-play

Ejemplo moderno: Drivers NVMe, cómputo en GPU/NPU



4. Sistema de Archivos

Abstracciones: Archivo (datos + metadatos) · Directorio (organización jerárquica) · Punto de montaje

Responsabilidades

- Organización en disco (FAT, NTFS, ext4, ZFS)
- Permisos y control de acceso
- Recuperación ante fallos (journaling)
- Caché de disco

Ejemplo moderno: Sistemas de archivos distribuidos (HDFS, Ceph)

5. Subsistema de Red



Responsabilidades

- Protocolos de red (pila TCP/IP)
- Interfaces de red (drivers de NIC)
- Firewall y filtrado
- API de sockets

Ejemplo moderno: Virtualización de red (OVS, eBPF)

6. Seguridad y Protección



Responsabilidades

- Autenticación (Linux PAM, Active Directory)
- Control de acceso (DAC, MAC — SELinux, AppArmor)
- Auditoría y logging
- Cifrado (dm-crypt, BitLocker)

Ejemplo moderno: TPM 2.0, Secure Boot, measured boot

Objetivos del Sistema Operativo

- Facilitar el uso al usuario (interfaz)
- Facilitar la programación al programador (máquina virtual extendida)
- Ejecutar programas eficientemente (gestor de recursos)

Conveniencia

Proveer un ambiente al usuario para ejecutar sus programas — por ejemplo, en computadoras personales.

Eficiencia

Incrementar la productividad de los recursos de producción del sistema.

Servicios del SO y Cómo se Relacionan



Silberschatz, A., Galvin, P. B., Gagne, G. (2014). Operating System Concepts. Wiley.

Ejercicio: Planificación de Procesos

- Cierta computadora dispone de un procesador; cada cierto tiempo llega un proceso, ejecutable según los tiempos de la tabla. Calculemos:
 1. El tiempo de término de los procesos, asumiendo que se ejecutan en secuencia.
 2. El tiempo de término, asumiendo que dos procesos se pueden ejecutar en paralelo.

Proceso	Tiempo Llegada	Tiempo Servicio
1	0	3
2	2	2
3	4	4
4	7	3
5	12	5

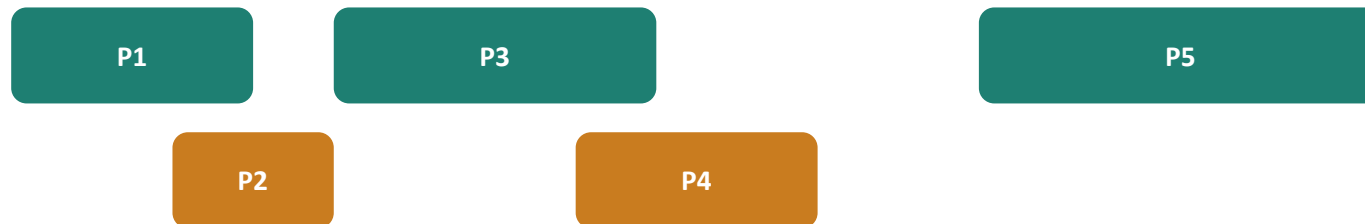
Ejercicio: Secuencial vs. Paralelo

Proceso	Tiempo Llegada	Tiempo Servicio
1	0	3
2	2	2
3	4	4
4	7	3
5	12	5

1. Ejecución secuencial



2. Ejecución en paralelo (2 procesadores)



Cada bloque inicia en su tiempo de llegada o cuando el procesador queda libre, lo que ocurra después.

Modo Dual de Operación

- Al compartir recursos, el SO debe asegurar que un programa no afecte a otro. ¿Cómo lo logra?
- Solución: un mecanismo de protección por hardware (bit) que habilita al menos 2 modos de ejecución — usuario (1) y sistema (0).
- Las instrucciones privilegiadas solo son posibles en modo sistema.
- Ante una interrupción, trap o excepción, el hardware cambia automáticamente a modo sistema.

CONEXIÓN CON UNIDAD 1

En la Unidad 1 (Bloque 2) ya vimos los modos kernel/usuario como parte del SO como intermediario. Aquí profundizamos en el mecanismo de hardware que los sostiene.

Protección por Hardware

E/S

Siempre instrucciones privilegiadas, en modo sistema.

Memoria

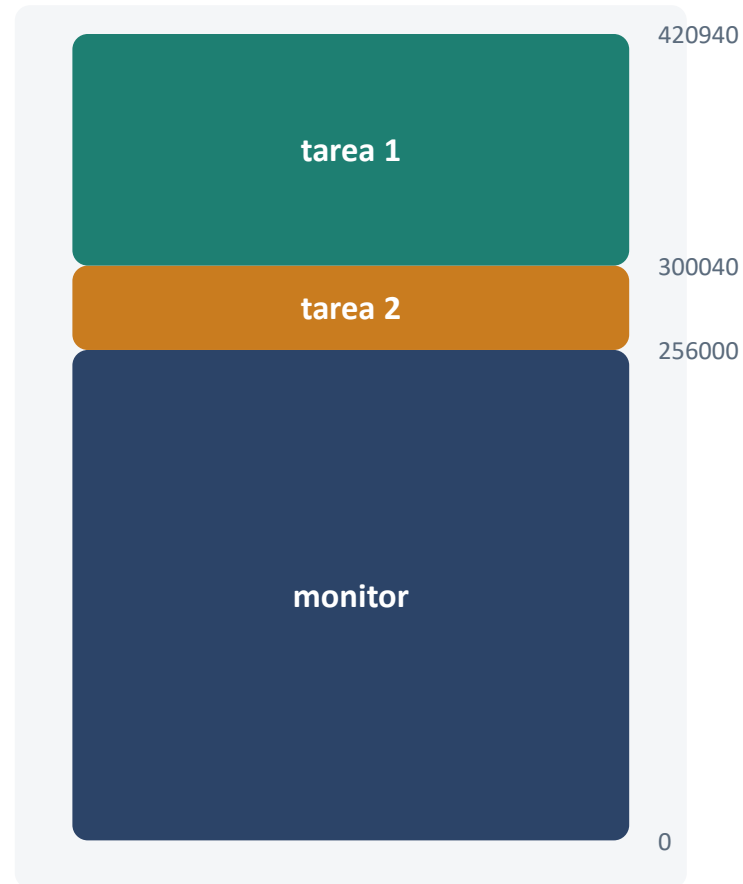
Registros base y límite por programa; los accesos fuera de rango originan un fallo de protección.

CPU

Un temporizador interrumpe periódicamente para devolver el control al SO y que este decida.

Protección por Hardware: Registros Base y Límite

- La tarea en ejecución solo puede direccionar dentro de [base, base+límite].
- Cualquier acceso fuera de ese rango genera un fallo de protección atendido por el SO.



Registro base: 300040 · Registro límite: 420940

Arquitectura General del Sistema

Si las instrucciones de E/S son privilegiadas, ¿cómo ejecuta el usuario la E/S? → Llamada al sistema.

- 1 El programa genera un trap (interrupción por software).
- 2 Se salta a ejecutar la RTI (Rutina de Tratamiento de Interrupción, un servicio del SO) y se cambia a modo sistema.
- 3 El SO verifica los parámetros, ejecuta la petición y devuelve el control en la instrucción siguiente.
- 4 El SO oculta las llamadas al sistema dentro de bibliotecas que se instalan junto con él.

ARQUITECTURAS

Arquitecturas de Sistemas Operativos

Estructura Simple (Monolítica)

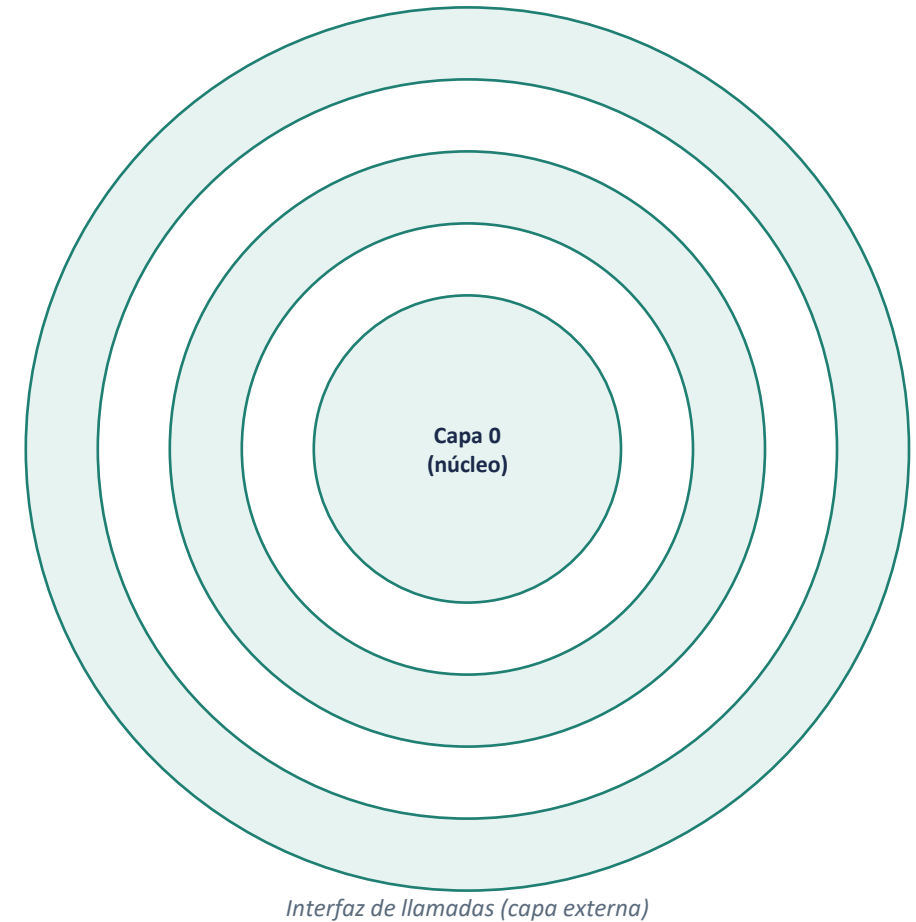
- Todo está entre las llamadas al sistema y el hardware.
- No está dividido en módulos; interfaces y niveles de funcionalidad no están separados.
- Escrito para una funcionalidad específica y hardware reducido.
- Monoproceso.



Ejemplo: MS-DOS

Enfoque por Capas

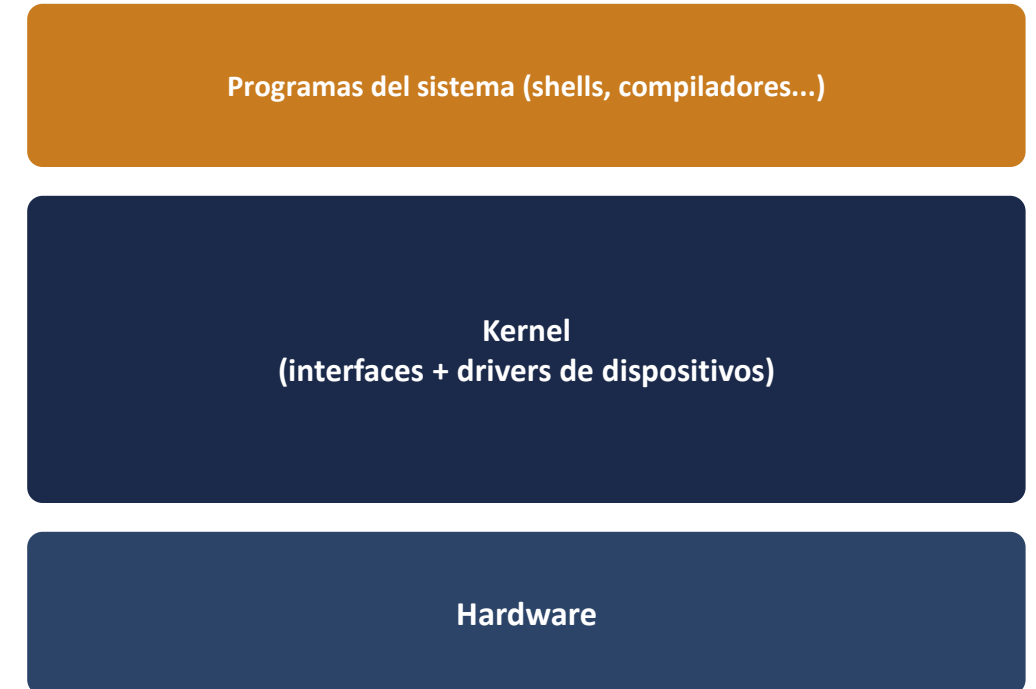
- Dividido en múltiples niveles (“capas de una cebolla”).
- El nivel más alto ofrece la interfaz de llamadas al sistema.
- Cada nivel usa únicamente servicios del nivel inferior y da servicio al superior.
- Riesgo: el orden de las llamadas entre capas puede generar problemas.



Ejemplo: Multics (Multiplexed Information and Computing Service, 1964)

Estructura UNIX Tradicional

- Consta de dos partes separables: el núcleo (kernel) y los programas del sistema.
- El kernel se separa en interfaces y controladores de dispositivos, ampliados a lo largo de la evolución de UNIX.
- Puede verse como un sistema en capas hasta cierto punto.



Sistemas con Micronúcleo (Microkernel)

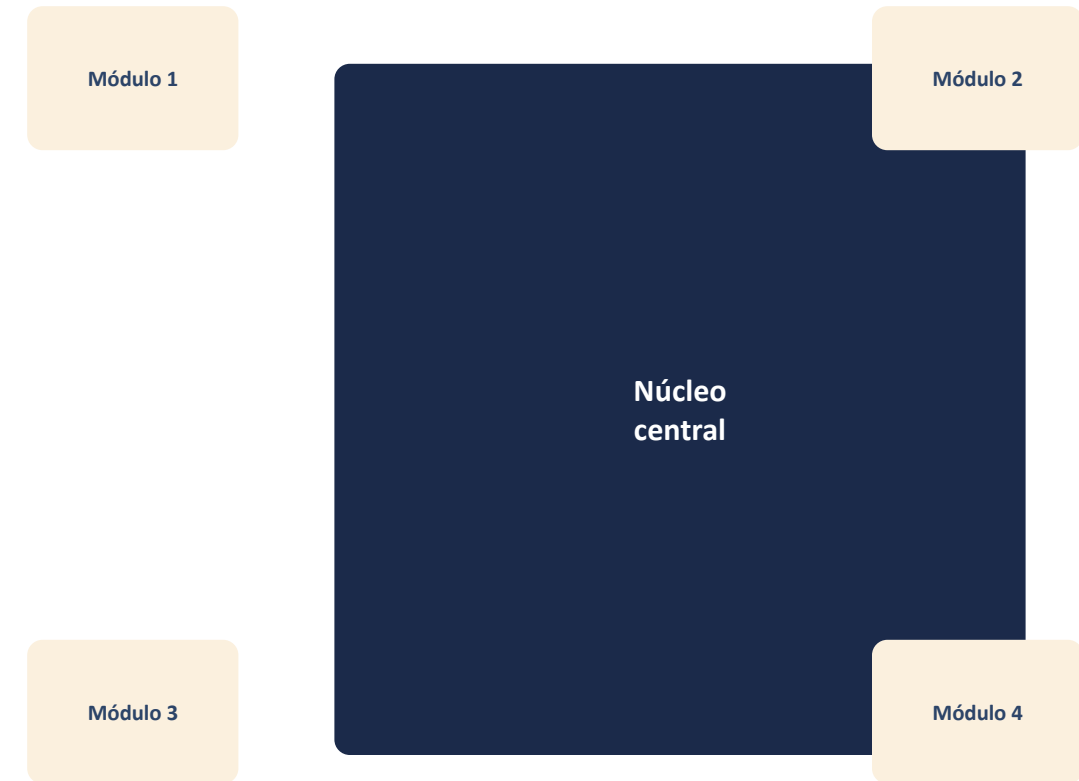
- Reducido a las funcionalidades mínimas imprescindibles.
- Los demás servicios se sacan fuera del núcleo, como servidores independientes.
- Comunicación vía paso de mensajes.
- Problema: una llamada puede necesitar muchos mensajes. Ventaja: piezas separables, camino hacia SO distribuido.

Ejemplo: QNX, MINIX 3



Sistemas Modulares

- Dividido en módulos con funcionalidad e interfaces claramente definidos.
- Modular y estructurado; los componentes se pueden sustituir por otros con el mismo interfaz.
- Muchas funcionalidades: soporta multiproceso y multiusuario.



Ejemplo: Linux (módulos de kernel cargables), Windows (drivers WDM)

Próximos Pasos

Próxima sesión (Semana 3)

- Estructura del Sistema Operativo — Máquinas virtuales
- Administración de Procesos: conceptos, comunicación entre procesos, threads
- Planificación del CPU y Sincronización de Procesos
- Presentación de la propuesta: Fases 2 y 3 Actividad en Equipos - Solución basada en Principios de los Sistemas Operativos

PUENTE HACIA EL PROYECTO PROGRAMADO I

Esta misma sesión continúa con el arranque de la Fase 2 (Conceptualización) y Fase 3 (Diseño Preliminar) la Actividad en Equipos - Solución basada en Principios de los Sistemas Operativos, y la entrega de las Preguntas de Comprobación individuales en Semana 3.

MUCHAS GRACIAS

Bibliografía

- Silberschatz, A.; Galvin, P.; Gagne, G. Operating System Concepts, 10th ed., Wiley, 2018.
- Stallings, W. Operating Systems: Internals and Design Principles, 9th Global Edition, Pearson, 2018.
- Stallings, W. Computer Organization and Architecture: Designing for Performance, 11th ed., Pearson, 2022.
- Tanenbaum, A.; Bos, H. Modern Operating Systems, 5th ed., Pearson, 2023.